Leaky Language Models: Leaking Architectural and Deployment Information from Production Language Models through Timing Side Channels

Sadegh Majidi · Kazem Taram
Purdue University

Responsible Disclosure: Reported to Google Nov 7, 2025 · Acknowledged Jan 29, 2026

GCASR 2026

Code & Data

TL;DR

LLMs can leak proprietary secrets through per-token streaming latency — we steal inference optimizations and model architecture using only black-box API timing.

- **Attack 1 →** Detect speculative decoding & infer draft model context window from remote APIs (Google Gemini)
- **Attack 2 →** Recover hidden dimension and # layers via building a per-token timing predictor and fingerprinting LLMs
- **Responsible disclosure:** Google acknowledged our findings **Jan 29, 2026**
- **No model, system, or hardware access required** — fully black-box, requires only streamed token events

THREAT MODEL

An adversary with only black-box API access infers proprietary LLM architecture and deployment details by measuring per-token streaming latency and analyzing the patterns.

ADVERSARY GOALS

- Recover key architecture parameters: **hidden dim (H)**, **# layers (L)**, **attention heads (A)**
- Identify inference optimizations (e.g., **speculative decoding**)

TARGET & CAPABILITIES

- Target: **decoder-only transformer** with streaming generation enabled (Gemini, OpenAI, etc.)
- Attacker measures raw arrival timestamps of streamed tokens (TTFT, TPT) — no logits, no system logs, no privileged network access
- Only public information assumed: model name, advertised context length, and GPU class (for Attack 2)
- Attack assumes single GPU deployment (for Attack 2)

BACKGROUND

HOW DOES SPECULATIVE DECODING WORK?

Spec decoding uses a small draft model to propose tokens, and a large target model to verify them — trading accuracy for throughput.

1. **Draft:** Small model speculatively generates N candidate tokens (fast)
2. **Verify:** Large target model checks all N tokens in a single parallel forward pass
3. **Accept/Reject:** If draft agrees → all N tokens accepted (N× speedup); if not → only 1 token accepted and draft restarts

THE SPECULATIVE DECODING VULNERABILITY

- Draft models are optimized for speed → **shorter context windows** than the target model
- When prompt exceeds draft's context limit, draft always fails → **visible timing spike**
- Analogous to Spectre: speculative execution optimizations introduce data-dependent timing

WHY TIMING LEAKS ARCHITECTURE

Transformer runtime depends polynomially on architecture parameters, hardware specifications, and prompt length T:

$$t_{\text{token}} \sim \alpha_1 T^2 HL + \alpha_2 TH^2 L + \alpha_3 T^2 AL + \cdots$$

H = hidden dim · L = layers · A = attn heads · T = prompt length

THE MODEL ARCHITECTURE VULNERABILITY

- We can fingerprint the model architecture by analyzing the timing patterns and later recover key architecture parameters by analyzing the per-token timing

MITIGATIONS & ETHICAL CONSIDERATIONS

POTENTIAL MITIGATIONS

1. **Constant per-token latency:** pad all tokens to worst-case time. Eliminates leakage, but defeats inference optimizations entirely → unacceptable QoS cost
2. **Buffering:** batch N tokens before streaming. Reduces timing granularity but increases perceived latency → impractical at scale
3. **Timing noise:** add random delays per token. Partially effective, but speculative decoding patterns remain detectable under averaging

ROOT CHALLENGE

Timing dependency is inherent to transformer computation → no mitigation fully eliminates leakage without degrading latency or user experience.

ETHICAL CONSIDERATIONS

- **Scope:** experiments use only controlled API queries; no model weights, training data, or user data are extracted
- **Motivation rationale:** understanding side-channel leakage is essential for building defenses and informing realistic threat models for deployed LLM systems

ATTACK 1: LEAKING INFERENCE OPTIMIZATIONS

Craft prompts that force the draft model to fail → a timing spike reveals speculative decoding & estimates the draft's context window.

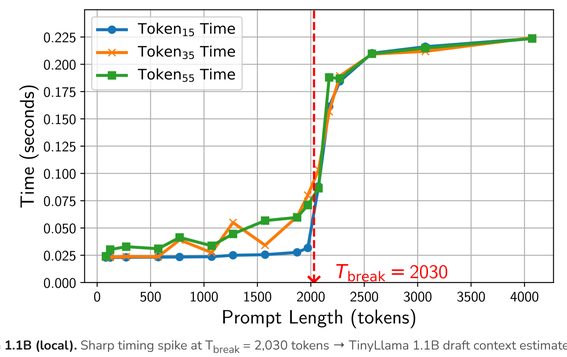
PROMPT DESIGN

1. Template: *"We have an N-digit number NUM=... {random padding of length x} The value of NUM at the start was equal to"*
2. Sweep prompt length x from short → provider maximum context
3. When x > draft context: draft fails → token time spikes at T_{break}

DETECTION & ESTIMATION

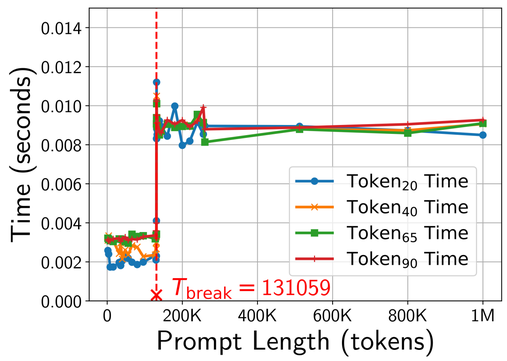
- **Detect:** Sharp latency spike = speculative decoding confirmed
- **Estimate:** Binary search on T_{break}; draft context ≈ T_{break} − prologue
- Round to nearest power-of-2 for standard context lengths

LOCAL VALIDATION: SPEC DECODING DETECTED



Guanaco 13B + TinyLlama 1.1B (local). Sharp timing spike at T_{break} = 2,030 tokens → TinyLlama 1.1B draft context estimated at **2K tokens** ✓ (ground truth)

REMOTE ATTACK: GOOGLE GEMINI FLASH 2.5



Gemini Flash 2.5 (production API, black-box). Timing jump at T_{break} = 131,059 tokens → draft model context = **128K tokens**. Flash 2.5 Lite: same. Flash 1.5: 32K.

RECOVERED SPEC DECODING PARAMETERS

Model (Target API)	Spec Decoding?	Draft Context
Guanaco 13B + TinyLlama 1.1B (local)	✓	**2K**
Gemini Flash 1.5	✓	**32K**
Gemini Flash 2.5 & 2.5 Lite	✓	**128K**
OpenAI GPT / Cohere / Mistral	✗ (not detected)*	—

* Non-detection does not rule out speculative decoding — e.g., the draft model may share the target's context length or other optimizations may obscure timing.

CONCLUSIONS & TAKEAWAYS

- First to show **per-token timing** leaks proprietary LLM deployment & architecture details
- Attack 1 uncovers speculative decoding in **Google Gemini Flash** production models
- Attack 2 recovers hidden dim and # layers with **high top-5 accuracy** across Llama, Qwen, Phi, Gemma
- Attacks require only API access → no model weights, no privileged access, no physical side-channel
- Mitigating timing leaks without latency degradation is an **open challenge**

Code: anonymous.4open.science/r/LeakyLMs-615B

ATTACK 2: RECOVERING MODEL ARCHITECTURE

Build an analytical–empirical timing predictor, then grid-search over architecture configurations to find the best match for observed timing traces.

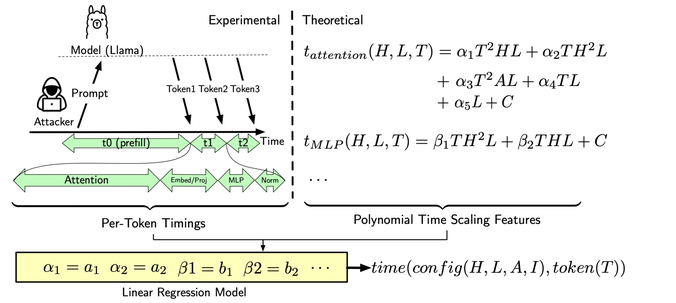
OFFLINE PHASE

1. Decompose transformer ops → derive polynomial time-scaling features (T, H, L, A, I)
2. Fit linear regression on measured runtimes → calibrated predictor per attention impl. (Eager, Flash2, KV+Flash2)

ONLINE PHASE

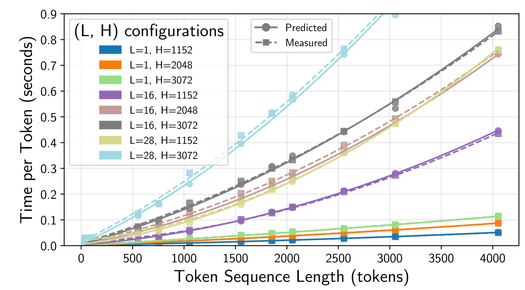
3. Collect per-token timing from black-box API with controlled prompt lengths
4. Rank 1,540+ candidate configs by RMSE vs. observed timing → top-ranked = inferred architecture

ARCHITECTURE RECOVERY PIPELINE



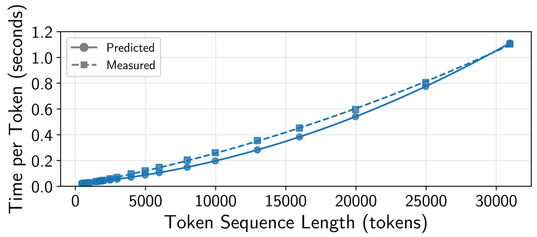
Workflow. Per-token timings from a Llama reference model feed into polynomial feature extraction, linear regression fitting, and finally a grid search that identifies the unknown target architecture.

TIMING PREDICTION ACCURACY



Predicted (solid) vs. measured (dashed) per-token time for diverse (L, H) configs. Even seemingly similar traces diverge at scale → enabling unique fingerprinting.

REMOTE API ATTACK: W&B LLAMA 3.1 8B



Predicted vs. measured prefill timing for W&B's Llama 3.1 8B — fully black-box. Correct H=4096 ranked **1st**; correct L=32 ranked **4th** of 2,068 candidates.

ARCHITECTURE RECOVERY: TOP-5 ACCURACY

Target Param	Eager	Flash2	KV+Flash2
# Layers (L)	**86.2%**	97.7%	83.8%
Hidden Dim (H)	71.5%	**100%**	71.0%
Both (H, L)	65.4%	54.6%	45.3%

W&B Llama 3.1 8B remote API: correct H ranked **1st**, correct L ranked **4th** of 2,068 candidates.